

AURA, rebuilt from memory.

This is the current-state rebuild dossier for AURA. If the repo is lost, a developer or agent should be able to recreate the app, backend, voice pipeline, memory system, reminders, iOS build path, Mac-to-Mac IPA handoff, and launch roadmap from this document.

AURA is a voice-first AI life journal: it listens, remembers quietly, recalls by date, notices emotional continuity, sets reminders, and speaks back through a resilient voice chain.

Expo 52

React Native 0.76.9 / JSC
/ New Architecture off

API :4000

Express backend bound
to 0.0.0.0 for iPhone LAN
testing

Full Recall

Raw archive, smart
memories, summaries,
emotional continuity

Scale Hooks

Quota ledger, brain
queue, Redis-ready
global semaphore



What AURA must do now.

The current app is not just a front-end. The experience depends on a mobile voice state machine, a backend orchestration layer, deterministic memory storage, emotional continuity context, and reminder delivery that changes based on foreground/background state.

Mobile shell

Expo iOS app named **AURA**, bundle id **ai.aura.journal.recall**, portrait-only, minimal grey-white UI, hidden controls, tap-to-speak interaction.

Backend brain

Express API on **0.0.0.0:4000**. It owns STT, NVIDIA/OpenRouter replies, TTS, memory, recall, reminder parsing, check-ins, and provider key rotation.

Voice loop

Tap records. Tap again transcribes with Groq Whisper. NVIDIA or OpenRouter replies. Backend synthesizes Gemini or Pocket voice. iOS speech is the final device fallback.

Memory promise

Archive every user and assistant turn as historical truth. Extract smart memories and emotional signals separately. Exact dates appear only when the user wants exact recall.

Reminder promise

Reminder intent beats memory recall. Foreground app shows reminder text and speaks it. Background or closed app shows a normal AURA notification banner.

Build promise

A second Mac can run `npm install && npm run ipa:ios`. The build script regenerates iOS, validates permissions, patches Pods, and exports an unsigned IPA.

Recreate this folder first.

The project is a root npm workspace with two apps. Keep the folder name unless deliberately creating a clone. This map names the files that define the current behavior.

```
build-a-mobile-application-called-second/
package.json           # npm workspaces + ipa:ios script
render.yaml            # Render web service config
scripts/build-ios-ipa.sh  # Expo prebuild, Pods patching, Xcode archive/package
docs/AURA_Product_Memory_Vision.pdf  # this recovery dossier + future launch plan
scripts/aura-beta-chaos.mjs  # backend-only real-user chaos simulation
apps/api/
  src/index.ts          # Express server, CORS, /health, request logs
  src/routes/capture.ts  # chat, memory, reminders, recall, TTS response
  src/routes/voice.ts    # /voice/transcribe and /voice/synthesize
  src/routes/reminders.ts # reminder listing/create/delete API
  src/services/openRouter.ts  # NVIDIA/OpenRouter fallback, brain queue, embeddings
  src/services/geminiVoice.ts # Gemini TTS, then Pocket TTS fallback
  src/services/pocketTts.ts  # kyutai-labs/pocket-tts via uvx, returned as PCM
  src/services/conversationArchive.ts # exact raw archive and date recall
  src/services/memoryStore.ts # smart memories + session summaries
  src/services/emotionalContinuity.ts # trends, restraint, relationships
  src/services/reminderParser.ts # natural time/check-in parsing
  src/services/reminderStore.ts # Supabase/local reminder persistence
  src/services/brainOrchestrator.ts # local queue + Redis-ready global semaphore
  src/services/usageLedger.ts # 100/day usage quota ledger
  data/*.json           # local fallback persistence when Supabase absent
apps/mobile/
  app.json              # bundle id, ATS/local network, mic, notifications
  src/App.tsx           # navigation, foreground reminders, idle finalize
  src/screens/CaptureScreen.tsx # tap/listen/think/reply screen
  src/screens/TypeInsteadScreen.tsx # typed chat path
  src/lib/api.ts         # API bridge + local fallback + provider headers
  src/lib/audio.ts       # PCM/WAV playback + loud iOS speech failsafe
  src/lib/reminderScheduler.ts # local notifications + due reminder cache
```

Rule: keep secrets in `apps/api/.env` or host env vars. Do not hardcode API keys into mobile code or this document.

The backend is the only place that should know provider keys.

The IPA should call a backend URL. The backend then talks to Groq, OpenRouter, Gemini, Pocket TTS, and optionally Supabase. Provider keys rotate server-side or can be sent as user-provided headers later, but never compiled into the IPA.

STT	Groq Whisper, model <code>whisper-large-v3-turbo</code> , endpoint <code>POST /voice/transcribe</code> .
LLM	Backend tries NVIDIA first for speed, then OpenRouter as backup. Current health shows NVIDIA 2 keys, OpenRouter 6 keys, NVIDIA primary <code>meta/llama-3.1-8b-instruct</code> , OpenRouter primary <code>qwen/qwen3-next-80b-a3b-instruct:free</code> .
Voice	Gemini TTS model <code>gemini-2.5-flash-preview-tts</code> , voice <code>Sulafat</code> ; then Pocket TTS; then iOS local speech.
Memory	Supabase Postgres if configured. Local JSON fallback under <code>apps/api/data</code> if Supabase is absent.
Network	Local IPA testing uses <code>http://192.168.1.7:4000</code> . Backend must bind <code>0.0.0.0</code> ; iPhone and Mac must share Wi-Fi.

```
# apps/api/.env shape
PORT=4000
HOST=0.0.0.0
APP_ORIGIN=*
OPENROUTER_API_KEYS=sk-or-v1-...,sk-or-v1-...
OPENROUTER_MODEL=qwen/qwen3-next-80b-a3b-instruct:free
OPENROUTER_FALLBACK_MODELS=openai/gpt-oss-20b:free,z-ai/glm-4.5-air:free
NVIDIA_API_KEYS=nvapi-...,nvapi-...
NVIDIA_MODEL=meta/llama-3.1-8b-instruct
NVIDIA_FALLBACK_MODELS=meta/llama-3.3-70b-instruct,mistralai/mistral-small-4-119b-2603
GROQ_API_KEY=gsk_...
GROQ_TRANSCRIPTION_MODEL=whisper-large-v3-turbo
GEMINI_API_KEYS=AIZA...,AIZA...
GEMINI_TTS_MODEL=gemini-2.5-flash-preview-tts
GEMINI_TTS_VOICE=Sulafat
POCKET_TTS_ENABLED=true
POCKET_TTS_COMMAND=uvx
POCKET_TTS_COMMAND_ARGS=pocket-tts,generate
POCKET_TTS_LANGUAGE=english
POCKET_TTS_DEVICE=cpu
POCKET_TTS_TIMEOUT_MS=14000
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
```

The routes that make the app feel alive.

The mobile app is intentionally thin. These routes are the behavior contract a rebuild must preserve.

01 GET /health

Returns service status and provider readiness for OpenRouter, Groq, Gemini/Pocket voice, memory persistence, and archive paths.

02 POST /voice/transcribe

Receives multipart audio as `audio`. Logs file presence, MIME, size, transcript length. Returns transcript or clear failure.

03 POST /capture

Main brain route. Saves user turn, detects reminders/check-ins, decides recall mode, builds memory context, calls OpenRouter, stores reply, returns text plus voice when available.

04 POST /voice/synthesize

Reminder/standalone voice endpoint. Input { `text` }. Output { `voice` }. Voice may be Gemini, Pocket, or `null`; reminder creation must never fail because TTS failed.

05 GET /reminders

Returns scheduled reminders/check-ins for the app UI and local cache. Mobile schedules iOS notifications from these reminders.

06 POST /memories/session/finalize

Creates compact session summaries after idle gaps so long conversations are recallable without dumping every raw turn into normal prompts.

Failure rule: OpenRouter/Gemini failures degrade gracefully. AURA should still create reminders, save archive turns, and produce deterministic responses for greetings/capability checks when possible.

How AURA hears and speaks.

The voice path is the soul of the app. The rebuild must preserve the state order and the fallback chain.

Start recording

Tap

CaptureScreen records AAC/M4A using Expo AV. UI shows **Listening** in soft grey. If the user taps during playback, current audio stops and recording starts cleanly.

Stop and think

Tap

Second tap stops recording, validates URI/file, uploads to `/voice/transcribe`, then sends transcript to `/capture`. UI shows **Thinking**.

Reply text first-class

AI

When `/capture` returns, reply text appears on the background like a premium AURA reply. No card shape. No robotic transcript dump.

Voice starts as close to text as possible

TTS

Normal replies use Gemini voice if returned. If missing or playback fails, mobile uses the same loud iOS speech helper as reminders.

Primary STT

Groq Whisper via backend. This prevents mobile from needing provider keys and allows backend logging/retries.

Normal voice fallback

Mobile `speakTextWithFailSafe` chooses a clear installed iOS voice, volume `1`, `playsInSilentModeIOS`, and a timeout unlock so the app does not stay stuck in speaking state.

The current TTS resilience layer.

Gemini TTS can hit quota/permission limits. The current live state shows Gemini voice keys cooling down, so Pocket TTS is now the working backend fallback before the phone ever falls back to iOS speech.

1. Gemini	Backend tries all active Gemini keys, with cooldown for 400/401/403/429. <code>/voice/synthesize</code> uses a fast race across active keys for reminders.
2. Pocket TTS	If Gemini cannot produce audio, backend runs <code>uvx pocket-tts generate</code> . It converts the resulting WAV into raw PCM when possible so current mobile playback handles it.
3. iOS speech	If Gemini and Pocket both fail, the app speaks locally through the loud shared iOS speech failsafe. It uses volume 1, rate 1.08, silent-mode playback, and the same helper for chat, Type Instead, and foreground reminders.

Reminder voice behavior:

- app active: wait for backend voice until the end of the scheduled minute, max 59s
- if backend voice arrives: set reminder text and play prepared audio together
- if backend voice does not arrive by deadline: set text and speak with iOS fallback
- mark delivered only after text is visible and voice attempt finishes

Backend voice endpoint response:

```
POST /voice/synthesize { "text": "Hey Rihan, time to drink water." }  
=> { "voice": { "audioBase64": "...", "mimeType": "audio/pcm;rate=24000", "provider": "pocket-tts" } }  
=> or { "voice": null } when all backend TTS paths fail
```

Important: Background/closed app voice is not promised. iOS notifications show a banner only. Custom notification audio is intentionally not required because it can break delivery.

Foreground speaks. Background notifies.

Reminders are split into deterministic creation, local notification scheduling, and foreground voice delivery. Reminder intent must always beat memory recall.

Foreground/open app

Every second, `App.tsx` checks cached due reminders. If the app is active and not recording/speaking, it prepares voice, switches home, shows the reminder as normal AURA text, haptics fire, then voice plays.

Background/closed app

`expo-notifications` schedules a normal AURA text notification. No voice. When the app opens after a fired banner, `markDueNotificationRemindersHandled` prevents duplicate foreground voice spam.

Check-ins

"Ask me if I'm still smoking weed after 30 minutes" becomes a `check_in` reminder with a question and guidance. The answer is linked back and saved into memory.

Stuck-speech protection

If a previous reply leaves `voiceBusy` stuck, the stale unlock clears after 20 seconds so due reminders can still run.

Expected user flow:

User: remind me to drink water after one minute

AURA: Done, Rihan. I'll remind you at 01:40. Please keep AURA open around then so I can speak it to you.

At due time while app is open:

Screen text: Hey Rihan, time to drink water.

Voice: Gemini AURA voice -> Pocket TTS -> iOS fallback

At due time while app is closed/backgrounded:

Notification: AURA / Hey Rihan, time to drink water.

No background voice promise.

Reliable recall is the core magic.

AURA has two layers: deterministic historical truth and derived intelligence. Never replace the archive with summaries. Summaries make recall fast; the archive keeps it honest.

Raw**Conversation archive**

Every user transcript/text and every AURA reply is saved with exact timestamp, timezone, session id, input mode, role, and searchable text. This powers date-wise recall.

Smart**Smart memories**

After each useful turn, **memoryAnalyzer** extracts summary, emotion, topics, goals, action signals, importance, tags, and memory layer.

Embed**Recall vectors**

When configured, Gemini embeddings use **gemini-embedding-001** and 1536 dimensions. If unavailable, keyword/recency fallback still works.

Idle**Session summaries**

After roughly 5 minutes idle, mobile calls **/memories/session/finalize**. Backend summarizes recent archive turns into a session-summary memory.

UX rule: normal chat saves silently. AURA says "I heard you. I saved this moment." only when the user explicitly says **save/remember/journal** this.

Exact when asked, subtle by default.

The major memory UX improvement is mode separation. AURA should not prove it remembers by dumping logs every turn.

Utility recall	Triggers on "what did I say", "when did I", "yesterday", "last week", "on May 14", "show history", exact timestamps, reminders, and explicit save commands.
Companion mode	Normal emotional conversation. AURA uses memory quietly: "You've seemed worn out lately," not "You said tired on May 3, May 5..."
Archive answer	For date recall, <code>conversationArchive.ts</code> retrieves raw turns by date range/timezone and builds a clean answer hint. If no entries exist, AURA says so honestly.
Smart context	For normal replies, relevant smart memories, session summaries, emotional continuity, and theme scores are injected as quiet context, not visible logs.

Examples:

"What did I say yesterday?" -> utility mode, timestamped archive answer.

"What did I say today about Swift?" -> utility mode, filtered raw archive + summary.

"I'm tired again." -> companion mode, maybe subtle continuity if repeated and cooldown allows.

"Remember this: I want to focus on fitness." -> explicit save, one acknowledgement, smart memory.

Memory that feels human, not creepy.

The emotional layer is derived context. It never overwrites historical archive truth. Its job is restraint, pattern awareness, and softer companion phrasing.

Signals

After capture, AURA detects soft labels like tired, stressed, sad, excited, calm, motivated, overwhelmed. Each signal has timestamp, confidence, topic, intensity, and source turn.

Theme scores

Repeated themes such as sleep, work, family, fitness, trading, money, study, relationship, loneliness, and goals gradually become more important.

Relationships

Lightweight person continuity tracks dad, mother, friend, girlfriend, cofounder-like mentions, tone, related topics, last seen, and recurrence.

Evolution

If the user says "I love trading" and later "I hate trading," the archive keeps both. Companion mode treats the theme as changed or emotionally mixed.

Restraint

Cooldowns prevent AURA from mentioning the same pattern repeatedly. Low-confidence or sensitive patterns stay silent unless directly relevant.

Natural phrasing

Prompt context says: use continuity quietly, no timestamp dumps, no "I am analyzing your memory," no forced therapy-bot behavior.

Goal: AURA should feel like a continuous presence that notices patterns over time, while still letting exact recall remain deterministic and trustworthy.

Minimal, alive, and quiet.

The approved direction is minimal grey-white, premium typography, hidden controls, and an alive orb/logo presence. The app should not feel like a dashboard.

Launch

Clean AURA waking screen: large spaced wordmark, WAKING, UP. It fades into the main screen without jump cuts.

Home

Soft grey-white background. AURA wordmark and floating orb/logo style. No permanent clutter. Hold/press reveals menu/type controls depending on current build behavior.

Tap flow

Tap starts listening. Tap again sends transcript to AURA. Thinking stays visible until reply is ready. Reply text appears directly on background.

Controls

Hamburger opens the drawer. Writing/type icon opens Type Instead. Drawer is now restyled to the same quiet grey-white AURA aesthetic instead of the older blue-heavy panel.

Do not regress: fallback visuals must never show the old ugly SVG/orb or a grey rectangle under the orb. Drawer/menu surfaces should feel like soft frosted grey-white glass: muted icons, black/grey AURA wordmark, subtle dividers, no loud blue styling.

Why Safari can work while the app fails.

Physical iPhone builds must not use localhost. Local HTTP requires iOS permissions and ATS allowances. The build script refuses to continue if required keys are missing from generated Info.plist.

API URL	EXPO_PUBLIC_API_URL=http://192.168.1.7:4000 for local IPA tests. Safari health check must pass on the iPhone.
Backend bind	app.listen(env.port, env.host) with HOST=0.0.0.0, not localhost.
ATS	NSAllowsLocalNetworking or NSAllowsArbitraryLoads must exist in generated Info.plist for local HTTP.
Permissions	Microphone, local network, notification permissions are declared/requested. Notifications are only for banner delivery, not background voice.

```
Mac backend test:
npm run dev --workspace apps/api
curl http://127.0.0.1:4000/health
curl http://192.168.1.7:4000/health
```

```
Phone test:
Open Safari on iPhone -> http://192.168.1.7:4000/health
Then launch AURA and test voice.
```

The IPA path that currently works.

The iOS folder is regenerated before build. Do not trust stale `apps/mobile/ios`. The script applies local compatibility patches for React Native 0.76 on newer Xcode.

01

Install dependencies

Run from repo root. Node 20+ required.

02

Expo prebuild

`npx expo prebuild --platform ios --no-install`. The script checks Info.plist for local network/ATS.

03

Pods patch

The script disables warning-as-error in third-party Pods, pins iOS deployment target, patches `fmt` consteval, and skips Hermes replacement for local unsigned IPA.

04

Xcode build

Builds Release for iPhoneOS with code signing disabled, packages Payload into `AURA-unsigned-sideloadly.ipa`.

Clean target Mac before copy:

```
cd ~/Downloads && mkdir -p "pre builds" && for x in build-a-mobile-application-called-second build-a-mobile-application-called-second.zip AURA-IPA-BUILD AURA build; do [ -e "$x" ] && mv "$x" "pre builds/${x}-${date +%Y%m%d-%H%M%S}"; done; rm -rf ~/Library/Developer/Xcode/DerivedData/AURA*; find ~/Library/Developer/Xcode/DerivedData -maxdepth 1 -name '*AURA*' -exec rm -rf {} +
```

After copy/unzip on target Mac:

```
cd ~/Downloads/build-a-mobile-application-called-second && npm install && npm run ipa:ios 2>&1 | tee aura-build.log
```

Expected output:

```
build/ios/AURA-unsigned-sideloadly.ipa
```

How we move builds to TESSERACT.

A local Codex skill now documents the Mac-to-Mac handoff ritual so future agents do not guess. It archives old zips/folders instead of deleting them.

Skill path

/Users/rihan/.codex/skills/aura-airdrop-ipa-handoff/SKILL.md

Sender Mac

Move old same-name zip/folder/build artifacts into **pre builds**, create a normal full zip with **ditto --keepParent**, then AirDrop to TESSERACT.

Receiver Mac

Before AirDrop, run the clean/archive one-liner in Downloads. After AirDrop, unzip the new zip, enter the project folder, and run the after-copy build command.

Human checkpoints

Wait for Rehan to confirm the right zip, AirDrop target, unzip result, and before pressing Enter on the target terminal when requested.

Latest known handoff: a normal ~203.9 MB zip was AirDropped to TESSERACT. The target Mac ran `npm install && npm run ipa:ios`, Xcode finished with `** BUILD SUCCEEDED **`, and Finder revealed `~/Downloads/build-a-mobile-application-called-second/build/ios/AURA-unsigned-sideloadly.ipa`.

Use this before trusting a rebuild.

AURA should pass these user-level scenarios after any rebuild. If one fails, fix that subsystem before adding features.

Health: Mac and iPhone Safari both open `http://192.168.1.7:4000/health`. Health shows OpenRouter/Groq/Gemini/memory/archive status.

Voice chat: tap -> Listening; tap -> Thinking; transcript reaches backend; reply text appears; Gemini/Pocket/iOS voice plays.

Type Instead: typed message reaches `/capture`, reply text appears, fallback voice uses the same loud helper.

Memory: say "my goal is to learn Swift this month"; ask "what did I say today about Swift?"; AURA recalls from archive with date context.

Silent save: normal chat does not spam "saved." Explicit "remember this" gives one clean acknowledgement.

Emotional continuity: repeated tired/stress messages produce subtle wording like "you've seemed worn out lately," not timestamp dumps.

Reminder foreground: "remind me to drink water after one minute," keep app open. At due time, text appears, haptic fires, voice plays Gemini/Pocket/iOS, no banner.

Reminder background: set a reminder, background/close app. Normal AURA notification banner appears on time. No background voice is expected.

Check-in: "ask me if I'm still smoking weed after one minute," answer yes/no, AURA responds supportively and saves result.

When something breaks, start here.

Most failures fall into four buckets: provider quota, LAN networking, stale mobile cache/state, or iOS notification/voice behavior.

Network request failed	Check backend is running on 0.0.0.0:4000 , phone is same Wi-Fi, Safari health works, and generated Info.plist allows local networking.
"AURA brain" failure	Usually OpenRouter key/model exhaustion. Add/rotate OPENROUTER_API_KEYS , use the free model route, restart backend, verify health key count.
No Gemini voice	Gemini TTS quota/permission may be exhausted. Backend should fall to Pocket TTS. If Pocket also fails, mobile iOS fallback should speak loudly via volume 1 and rate 1.08.
Reminder created but silent	Check Xcode logs for [AURA reminders] : due reminder found, preparing voice, visible text set, provider used, fallback used, delivered.
Foreground duplicate after banner	markDueNotificationRemindersHandled should mark reminders delivered when the app opens after a fired iOS notification.
Robotic recall	Verify recall mode: exact timestamps only in utility recall. Companion mode must use continuity context quietly.

```
Useful commands:
screen -ls | grep aura-api
lsof -nP -iTCP:4000 -sTCP:LISTEN
curl http://127.0.0.1:4000/health | python3 -m json.tool
curl http://192.168.1.7:4000/health | python3 -m json.tool
npm run typecheck --workspace apps/mobile
npm run build --workspace apps/api
npm run beta:chaos
npm run beta:scale -- --stage=1 --users=10 --messages=5
npm run ipa:ios 2>&1 | tee aura-build.log
```

From zero folder to current AURA.

This is the shortest faithful path back to the current state.

Create workspace

- 1 Root npm workspace with `apps/api` and `apps/mobile`. Use Node 20+, TypeScript, Expo SDK 52, RN 0.76.9, JSC, New Architecture disabled.

Build backend

- 2 Express with CORS/logging, `/health`, `/capture`, `/voice/transcribe`, `/voice/synthesize`, `/reminders`, `/memories/session/finalize`.

Add memory truth

- 3 Raw archive first, then smart memory extraction, embeddings, session summaries, date recall, companion/utility recall decision, emotional continuity as derived context.

Add voice resilience

- 4 Groq STT, OpenRouter replies, Gemini TTS, Pocket TTS PCM fallback, iOS speech failsafe. Treat empty/generic LLM replies as retry/failure.

Add reminders

- 5 Parse natural reminders/check-ins server-side. Store reminders. Mobile schedules iOS banners and runs foreground voice delivery loop every second.

Ship iOS

- 6 Configure ATS/local network/mic/notifications. Regenerate iOS on every IPA build. Patch Pods for Xcode 26/RN 0.76. Export unsigned IPA for Sideloadly.

North star: AURA should feel less like a chatbot with logs and more like a continuous presence that quietly understands patterns over time.

The backend-only real-user chaos test passed 35/35.

AURA now has a repeatable backend beta test that behaves like a messy real user without touching Rehan's real memories. It sends generated voice through Groq STT, runs direct capture calls for interruptions and partial thoughts, forces controlled failures, and verifies memory, reminders, recall, emotional continuity, and voice fallback behavior.

Run id	aura-beta-chaos-1778901320300, isolated user id. Reports saved to /tmp/aura-beta-chaos/report.json and /tmp/aura-beta-chaos/summary.md.
Score	35/35 passed. Average latency 6855ms, max latency 12057ms, over-25s timeout risk 0.
Voice split	Pocket TTS returned audio in 33 cases. The only none voice cases were intentional chaos paths: simulated empty STT and forced all-backend-voice failure.
Quality gates	Reminder quality 7/7, recall quality 7/7, emotional continuity quality 6/6. No unexpected "couldn't connect to brain" fallback appeared.
What changed	Added <code>npm run beta:chaos</code> , dev-only chaos headers for <code>aura-beta-chaos-*</code> users, beta-only test timestamps, fast deterministic relationship recall, and shorter voice text for Pocket TTS reliability.

Current intended voice chain:

Normal chat: Gemini TTS -> Pocket TTS -> iOS speech

Foreground reminder: Gemini TTS -> Pocket TTS -> iOS speech

Background/closed reminder: normal AURA text notification only

Backend beta command:

`npm run beta:chaos`

Test coverage:

- generated WAV voice -> /voice/transcribe -> /capture -> memory/reminders/voice
- direct capture for interruptions, partial STT, rapid-fire messages
- forced brain failure, Gemini failure, Pocket failure, empty STT, memory delay
- date recall, relationship recall, reminder recall, explicit-save recall
- companion emotional continuity without timestamp spam

Important: These chaos controls are local/dev-only. They activate only for isolated `aura-beta-chaos-*` users with explicit `x-aura-chaos` headers, so normal app usage is not affected.

What is ready now, and what is still local.

The current repo is no longer just a prototype folder. It now has production-readiness hooks: quota accounting, brain queue metrics, scale runners, provider fallback, and health visibility. But it still runs as a local Mac backend until Supabase, Redis, and public HTTPS are connected.

Ready in code

/capture, /voice, /reminders, /usage, /brain/metrics, /health, memory archive, emotional continuity, reminder parsing, and TTS fallback are implemented.

Daily quota

DAILY_MESSAGE_LIMIT=100 is wired through a usage ledger. Local fallback writes to apps/api/data/daily-usage.json; Supabase uses increment_daily_usage.

Brain queue

Local beta uses BRAIN_MAX_CONCURRENCY=5. Production can set BRAIN_GLOBAL_MAX_CONCURRENCY=50 and REDIS_URL for cross-server semaphores.

Scale tests

beta:chaos, beta:concurrent, beta:brain-burst, beta:interrupt-storm, beta:scale, and beta:real-user exist to test memory, reminders, voice, queues, Brain Lab routing, replay, and no-voice structure load.

Current backend

Runs on the Mac at 0.0.0.0:4000. A detached screen session named aura-api can keep it alive for phone testing.

Not production yet

No public HTTPS URL, no live Supabase project, no Redis service, no production VPS, no App Store/TestFlight backend URL, and no real user account/privacy controls yet.

```
Production readiness knobs now recognized by the backend:
AURA_ENV=beta|production
DAILY_MESSAGE_LIMIT=100
BRAIN_MAX_CONCURRENCY=5
BRAIN_GLOBAL_MAX_CONCURRENCY=50      # production target
BRAIN_QUEUE_MAX_DEPTH=500           # production target
BRAIN_QUEUE_WAIT_MS=18000
BRAIN_JOB_TIMEOUT_MS=20000
REDIS_URL=redis://...               # required for multi-server global queue
REDIS_KEY_PREFIX=aura
SUPABASE_URL=https://...
SUPABASE_SERVICE_ROLE_KEY=...
```

How AURA launches from 1 user to 1,000 users.

AURA should not jump from a Mac backend directly to public launch. The right path is controlled: personal testing, private alpha, always-on beta, then TestFlight/App Store with cost controls.

Now: personal Mac beta

0

Keep using the installed IPA against the Mac backend. Validate voice, memory, reminders, recall, and emotional continuity daily. This is free-ish but requires the Mac online.

Private remote alpha

1

Create Supabase, deploy backend behind HTTPS, set provider keys as server env vars, rebuild IPA to call the public API. Keep users to Rehan and 2-5 trusted testers.

Always-on beta

2

Move API to a VPS/Oracle-style always-on machine. Install Node, Pocket TTS runtime, PM2/systemd, Caddy/Nginx HTTPS, log rotation, and uptime monitor.

50-concurrent day-1 backend

3

Add Redis and set `BRAIN_GLOBAL_MAX_CONCURRENCY=50`. API nodes stay stateless; brain and TTS workers scale separately. Reminders and deterministic recall bypass LLM whenever possible.

1,000-user launch

4

Use low-cost paid brain models, Pocket voice by default, capped Gemini premium voice, Supabase for all memory, Redis queue, per-user quota, cost dashboard, and graceful fallback.

Operating rule: launch only when AURA can survive backend restarts, provider quota failures, reminder bursts, and memory recall without cross-user leakage.

How AURA operates at 1,000 and then 10,000 users.

The subscription idea is **\$15/month** with **100 user messages/day**. The product must control AI usage so revenue covers LLM, STT, TTS, memory storage, and infrastructure.

Core assumption	100 user turns/day means about 3,000 LLM calls per user/month. Average context target is around 1,200 input tokens and 150 output tokens per reply.
Recommended voice mix	Pocket TTS primary, Gemini premium voice capped around 25% of replies, iOS speech emergency only. Gemini voice for every reply hurts margin.
1,000 users	Expected API cost about \$3.6k-\$4.0k/month , infra about \$600-\$900/month , total about \$4.2k-\$4.9k/month . Gross revenue is \$15k/month .
10,000 users	Expected API cost about \$36k-\$40k/month , infra about \$4k-\$7k/month , total about \$40k-\$47k/month . Gross revenue is \$150k/month .
Scaling strategy	Do not scale as one giant server. Split API nodes, brain workers, TTS workers, background memory jobs, Redis queues, and Supabase separately. Autoscale by queue depth and provider latency.
Hard guardrail	Do not sell unlimited AURA at \$15. Enforce 100 turns/day, track cost per user, and degrade heavy users to cheaper models/Pocket voice/summarized responses before margin breaks.

Recommended production architecture:
iOS App → Cloudflare/WAF → Load Balancer → Stateless API Nodes
API → Redis queues + global brain semaphore
API → Supabase Postgres + pgvector
Brain Queue → LLM workers, max 50 active at day-1
Voice Queue → Pocket/Gemini TTS workers
Background Workers → summaries, embeddings, cleanup, analytics
Observability → logs, metrics, alerts, cost dashboard

The remaining work before real users.

The repo is ready for serious beta hardening, but the public launch requires external infrastructure and trust work. This is the decision-complete checklist for the next operator or agent.

1. Supabase production: apply schema, verify memories, conversation_turns, reminders, daily_usage, emotional continuity tables, backups, export/delete plan.

2. Redis: provision Redis, set REDIS_URL, set BRAIN_GLOBAL_MAX_CONCURRENCY=50, verify /brain/metrics shows Redis global semaphore.

3. Public backend: deploy Node API to VPS/VM with HTTPS, env vars, PM2/systemd, log rotation, health checks, and Pocket TTS runtime.

4. IPA/TestFlight: rebuild app with public HTTPS API URL, verify STT, chat, reply voice, reminders, memory recall, foreground/background behavior.

5. Operations: add alerts for backend down, Supabase unreachable, Redis unreachable, all LLM keys cooling, Gemini/Pocket failures, reminder failures, and queue saturation.

6. Product trust: add privacy policy, support email, memory pause/delete/export controls, clear consent that AURA saves conversations to remember.

7. Beta limits: start with 3-5 active testers, then 25, then 100. Do not open 1,000 users until scale tests pass with Redis/Supabase on real infra.

Final north star: AURA should feel like a continuous presence with a real memory, not a chatbot proving it has logs. Reliability, restraint, and trust matter more than raw feature count.

The eval system now protects intelligence, scale, and cost.

AURA now has a cost-safe backend evaluation framework called **Brain Lab**. It tests conversation quality, memory routing, cross-user isolation, drift, latency, queue behavior, and cost risk without turning every simulated message into a paid LLM call.

Single-user intelligence

`npm run beta:real-user -- --stage=single` runs one evolving 40-message user audit. Latest pass: router accuracy **97.78%**, recall score **100%**, drift **0%**, cross-user leaks **0**.

100-user mixed mode

`--stage=100 --mode=mixed --real-users=15 --llm-max-calls=600` samples only selected users and critical paths. It tests distribution without a 4,000-call cost explosion.

1000-user structure mode

`--stage=1000 --mode=structure` makes zero LLM, STT, TTS, or external embedding calls. It tests routing, memory writes, reminder logic, queue stability, CPU/RAM, and leakage.

Sampled scale intelligence

`--stage=1000 --mode=sampled --critical-only=true --llm-max-calls=300` validates a tiny critical-path brain sample under load while keeping provider cost bounded.

Replay cache

`--replay-cache=write/read` stores normalized replies and replays them for mock users. This makes regression runs repeatable and debuggable.

Failure injection

`--failure-profile=light|medium|heavy` uses test-only chaos headers to inject brain delay, brain failure, and memory delay for fake users only.

Brain Lab commands:

```
npm run beta:real-user -- --stage=single
npm run beta:real-user -- --stage=100 --mode=mixed --real-users=15 --llm-max-calls=600 --replay-cache=write
npm run beta:real-user -- --stage=100 --mode=mixed --real-users=15 --llm-max-calls=600 --replay-cache=read
npm run beta:real-user -- --stage=1000 --mode=structure
npm run beta:real-user -- --stage=1000 --mode=sampled --critical-only=true --llm-max-calls=300
npm run beta:real-user -- --stage=100 --mode=mixed --failure-profile=medium --llm-max-calls=300
```

Smoke verification completed:

```
- 100 mixed: pass, 2 real calls / 18 theoretical, voice jobs 0
- replay read: pass, 31 hits / 0 misses
- 1000 structure: pass, 0 real LLM calls, voice jobs 0
- failure injection: pass, degradation shape smooth slowdown
```

Safety rule: real-call caps are the source of truth. Percent sampling only decides which requests are eligible. Any cross-user leak, backend crash, queue explosion, voice job during no-voice mode, or LLM calls above cap is a hard fail.